

Livrables — Mardik v2

Brief « Mardik — livrer et piloter la nouvelle version » (création, phase 4 : chaîne LLMOps & observabilité).

Application	https://mardik-api.onrender.com/ — client · pilotage · API
Dépôt	https://github.com/M-CHADLI/mardik-api-mlops

Ce dossier contient les **cinq livrables du brief, et rien d'autre** : ni les fichiers fournis avec le squelette, ni les tests, ni l'outillage. Les fichiers sont des copies ; la source dans le dépôt fait foi, et `python -m scripts.assembler_livrables` les régénère (`INSTANTANE.txt` donne le commit de référence).

1. Dossier de conception

besoin + contrat d'API + schéma de chaîne + schéma de boucle + tableau de pilotage

`1-dossier-de-conception/` — un document, les cinq pièces dans l'ordre :

§	Pièce
1	Synthèse du besoin — fonctionnel, contraintes, hors périmètre (traduction et chatbot, et pourquoi)
2	Langage du domaine — <i>contrat tronqué, analyse non fiable</i> , ce que « confiance » veut dire pour un juriste qui décide
3	Choix d'outils non triviaux
4	Contrat d'API — <code>/v1</code> intangible, <code>/v2</code> , la gateway, codes d'erreur, règle de versionnage
5	Schéma de la chaîne LLMOps — gates bloquants → build → artefact étiqueté → canary
6	Schéma de la boucle d'observabilité — signaux → seuils → les trois rétroactions
7	Tableau de pilotage — signal × seuil × rétroaction × trace
8	Questions ouvertes du brief et réponses retenues

2. L'API v2 du modèle

documents longs + score de confiance + erreurs explicites, v1 intacte

`2-api-v2/`

Fichier	Ce qu'il porte
<code>app__api_v2.py</code>	le contrat <code>/v2</code> , l'orchestration map-reduce, les appels de sections parallélisés
<code>app__pipeline__decoupage.py</code>	documents longs : découpage par article, invariant « rien n'est perdu »
<code>app__pipeline__extraction.py</code>	un appel LLM par section, sortie JSON contrainte
<code>app__pipeline__consolidation.py</code>	fusion et dédoublonnage des clauses
<code>app__pipeline__confiance.py</code>	score de confiance : confiance déclarée × ancrage de l'extrait
<code>app__gateway.py</code>	routage canary, en-tête <code>X-Mardik-Version</code>
<code>models__v2__config.yaml</code>	le bundle — <i>la version</i> , au sens où on l'étiquette et la rollback

Erreurs explicites : 422 (corps invalide), 503 (fournisseur indisponible, aucune version active), 500 nommé et journalisé — jamais de 500 muet.

v1 intacte : `app/api_v1.py` n'est **pas** dans ce dossier, et c'est le propos — il est fourni et non modifié. La non-régression est vérifiée à chaque fusion par `scripts/client_v1.py` et le test `test_client_v1_fonctionne`.

3. La chaîne LLMOps

fichier yml : gates bloquants, artefacts étiquetés, déploiement canary

3-chaine-llmops/

Fichier	Ce qu'il porte
<code>.github__workflows__llmops.yaml</code>	le fichier yml : lint → tests → gate d'évaluation → build → gate modèle réel → étiquetage → canary
<code>eval__run_eval.py</code>	le gate : note, latence P95, coût, dispersion entre passes
<code>ops__deploy.py</code>	étiquetage, canary , promotion, rollback, surveillance

Ce qui bloque : `publier()` rejoue le gate et refuse l'étiquetage s'il échoue. Aucun chemin vers le registre ne contourne la vérification, ni par la CI, ni par la console.

Artefact étiqueté : un dossier `ops/registry/vX.Y.Z/` avec le bundle figé et son manifeste — commit, empreinte SHA-256 du bundle, note d'éval, seuil, latence, coût, confiance de référence, date.

4. L'observabilité

tableau de bord + les trois boucles de rétroaction + le journal de pilotage

4-observabilite/

Fichier	Ce qu'il porte
<code>ops__dashboard.py</code>	tableau de bord : trafic v1/v2, latence P50/P95, taux d'erreur, coût, et la distribution du score (tranches métier + P10/P50/P90)
<code>eval__enrichissement.py</code>	boucle 3 : capture des cas peu fiables → validation humaine → versement au jeu d'éval → rejeu par le gate
<code>eval__anonymisation.py</code>	pseudonymisation avant toute écriture disque (un cas versé part dans Git et dans la CI)
<code>docs__journal-de-pilotage-exemple.jsonl</code>	le journal de pilotage : un cycle complet réel

Les boucles 1 et 2 vivent dans `ops/deploy.py`, livré au §3 pour ne pas le dupliquer :

#	Boucle	Fonction	Déclencheur
1	Rollback sur signal	<code>surveiller()</code>	confiance < 0,70, erreurs > 10 %, ou P95 > 8 s — automatique
2	Promotion canary pilotée	<code>promouvoir_si_conforme()</code>	confiance ≥ référence du gate – marge, sur ≥ 30 mesures → 10 → 30 → 100 %
3	Enrichissement du jeu d'éval	<code>eval/enrichissement.py</code>	analyse servie sous le seuil de confiance

Les trois sont déclenchables depuis le tableau de bord en ligne.

Le journal livré couvre un cycle entier, dans l'ordre :

```
publication · canary · ajustement_seuil · capture_faible_confiance ×6  
· promotion_canary · enrichissement_jeu_evaluation · rollback
```

avec, sur la dernière ligne, le motif qui a déclenché le retour arrière — `score de confiance moyen 0.523 < 0.7` — et l'état avant/après.

5. Un client via un frontend accessible via un lien

<https://mardik-api.onrender.com/>

5-frontend/

Fichier	Ce qu'il porte
<code>web__index.html</code>	le client juriste : coller un contrat → clauses et fiabilité
<code>web__tableau-de-bord.html</code>	le pilotage : trafic, distribution, cas à relire, journal
<code>web__commun.css</code>	la feuille de style
<code>app__frontend.py</code>	les routes et l'API du frontend
<code>render.yaml</code>	le déploiement qui produit le lien

Le client tape sur `/analyse` (la gateway), pas sur `/v2` : il voit donc ce que voit un vrai client, canary compris. L'interface parle la langue du métier — *fiable, à vérifier, à reprendre* — et non « score ».